

Clase 111 — Seguridad de APIs GraphQL

Parte: **4 — Seguridad de aplicaciones web** · Fuente: *PortSwigger Research / OWASP* 🕒 Duración estimada: **100 min** · Nivel: **Avanzado**

🎯 Objetivo

Auditar la seguridad de **APIs GraphQL**, cada vez más comunes. GraphQL cambia el modelo (un único endpoint, consultas flexibles) e introduce riesgos propios: introspección, over-fetching, ataques de complejidad (DoS), y los mismos problemas de autorización que REST, a menudo peor gestionados.

⚠️ **Ética:** solo en APIs propias/autorizadas.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explorar** el esquema con introspección y herramientas de GraphQL.
2. **Detectar** fallos de autorización (IDOR/BOLA) en queries y mutations.
3. **Explotar** over-fetching y batching para eludir límites.
4. **Provocar** DoS por consultas anidadas/complejas.
5. **Recomendar** desactivar introspección en prod y limitar complejidad.

📖 Temas

#	Tema	Por qué importa
1	Modelo GraphQL	Cambia la superficie
2	Introspección del esquema	Mapa completo de la API
3	Queries vs. mutations	Lectura y escritura
4	Autorización en resolvers	Donde suele fallar
5	Batching y alias	Bypass de rate limit
6	Ataques de complejidad (DoS)	Disponibilidad
7	Defensa: límites y authz	Cierre del fallo

📖 Definiciones y características

- **GraphQL:** lenguaje de consulta con un único endpoint donde el cliente define qué datos quiere. Característica: flexibilidad que amplía la superficie.

- **Introspección:** capacidad de consultar el propio esquema. Característica: revela todos los tipos y campos; peligrosa en producción.
- **Resolver:** función que resuelve un campo. Característica: si no valida authz, hay IDOR/BOLA.
- **Batching:** enviar múltiples operaciones en una petición. Característica: puede eludir rate limiting (p. ej. fuerza bruta).
- **Alias:** renombrar campos para repetir una consulta muchas veces. Característica: amplifica ataques en una sola petición.
- **Ataque de complejidad:** consulta profundamente anidada que agota recursos. Característica: DoS sin gran volumen de peticiones.

Herramientas y preparación

- **GraphQL/Altair** o **Burp** con extensiones **InQL**.
- **DVGA (Damn Vulnerable GraphQL Application)** como lab.
- **clairvoyance** para reconstruir esquema si la introspección está deshabilitada.

```
git clone https://github.com/dolevf/Damn-Vulnerable-GraphQL-Application
docker run -t -p 5013:5013 dolevf/dvga
```

Laboratorio guiado

 Solo en labs propios.

1. Localiza el endpoint GraphQL (`/graphql` , `/api/graphql`) y prueba la **introspección**:

```
{ __schema { types { name fields { name } } } }
```

2. Con InQL, genera el mapa de queries y mutations disponibles.
3. Prueba **IDOR/BOLA**: consulta un objeto por ID que no es tuyo (`user(id: 2){ email }`).
4. Ejecuta una **mutation** sensible sin el rol adecuado (autorización rota).
5. Usa **alias/batching** para repetir un login muchas veces en una sola petición (fuerza bruta sin rate limit).
6. Lanza una **consulta profundamente anidada** para evaluar el riesgo de DoS por complejidad.
7. Documenta introspección, authz rota y el vector de DoS.

Ejercicios

1. Reconstruye el esquema de DVGA con introspección y con clairvoyance (deshabilitada).
2. Explota un IDOR en una query y otro en una mutation.
3. Usa alias para hacer fuerza bruta en una petición.
4. Diseña una consulta anidada que dispare un ataque de complejidad.
5. Explica por qué desactivar introspección en prod reduce (pero no elimina) el riesgo.
6. Propón límites de profundidad/complejidad y authz por resolver.

Reto verificable

En DVGA, obtén el **esquema por introspección**, explota una **autorización rota** (query o mutation) y demuestra un **bypass de rate limit** con batching/alias. **Criterio de aceptación:** entregas el esquema, la operación no autorizada con su evidencia y el batching que elude el límite, más la defensa (introspección off, authz por resolver, límites de complejidad).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Introspección deshabilitada	Usa clairvoyance para inferir el esquema
Query da error de tipo	Sintaxis GraphQL; ajusta campos/argumentos
Mutation rechazada	Requiere rol; busca otra vía o documenta fortaleza
Batching ignorado	El servidor no soporta batch; usa alias
DoS no reproducible	Hay límite de profundidad; documenta la defensa

? Preguntas frecuentes

? **¿GraphQL es más inseguro que REST?** No inherentemente, pero su flexibilidad y la introspección amplían la superficie y a menudo la autorización está peor implementada.

? **¿Basta con desactivar la introspección?** Ayuda, pero un atacante puede inferir el esquema (clairvoyance). La defensa real es authz por resolver y límites de complejidad.

? **¿Por qué el batching es peligroso?** Permite empaquetar muchas operaciones en una petición, eludiendo rate limits diseñados para contar peticiones, no operaciones.

Referencias

- PortSwigger GraphQL API vulnerabilities: <https://portswigger.net/web-security/graphql>
- OWASP GraphQL Cheat Sheet.
- DVGA: <https://github.com/dolevf/Damn-Vulnerable-GraphQL-Application>
- InQL: <https://github.com/doyensec/inql>

Siguiente clase

Clase 112 - Web cache poisoning y HTTP request smuggling